

P3441R0

Rename `simd_split` to `simd_chunk`

Published Proposal, 2024-10-15

This version:

<http://wg21.link/P3441R0>

Author:

[Daniel Towner](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

The behaviour of `simd_split` is inconsistent with `std::ranges::views::split`, but is consistent with the behaviour of `std::ranges::views::chunk`. We propose to rename `simd_split` to `simd_chunk` to reflect this similarity in behaviour. We also propose that `simd_chunk_n` is provided as a convenience for a common use case.

Table of Contents

1 Motivation

2 Wording

2.1 Rename split to chunk in overview

2.2 Add `simd_chunk_n` to overview

2.3 Add description of `basic_simd` and `basic_simd_mask` creation [`simd.creation`]

§ 1. Motivation

The `std::simd_split<T>` function takes a `basic_simd` object and breaks it down into a tuple of as many objects of type `T` that it can, and maybe one remainder object. An example illustrates this:

```
simd<float, 19> x;
using IntoType = simd<float, 8>;

auto t = simd_split<IntoType>(x);
// get<0>(t) will be of type simd<float, 8>
// get<1>(t) will be of type simd<float, 8>
// get<2>(t) will be of type simd<float, 3> - the remainder
```

If the original type is perfectly divisible into type `T` then an `array<T>` is returned instead of a tuple of different sized `basic_simd` objects.

The behaviour `simd_split` is virtually identical to that of `std::ranges::views::chunk` and `std::ranges::chunk_view`. They take a view and a number `n` and produce a range of views (the chunks) of the original view, such that each chunk, except maybe the last one, has the size `n`.

In contrast, the more closely named `std::ranges::views::split` has a different behaviour to `simd_split`. The ranges version of `split` takes an input range and a delimiter value, and generates a range of views split on the delimiter. For example, the string "This,is,a,list", when split by the comma value would generate a range containing 4 views: "This", "is", "a", "list".

We propose that the `simd_split` function is renamed to `simd_chunk` to make its behaviour consistent with the existing range/view counterparts.

Note that a common use case for `simd_split/simd_chunk` is to break a larger `basic_simd` object into smaller native-sized pieces to call target-specific intrinsics. In this case the size of the chunk is known, but the user has to supply a complete type as the first template parameter to `simd_chunk`:

```
simd<float, 19> x;

// Break into AVX2-sized pieces:
auto t = simd_chunk<simd<float, 8>>(x);
```

In addition to this style we also propose that a short-hand is provided called `simd_chunk_n`, whose purpose is to make it easier and more readable to supply the new size:

```
simd<float, 19> x;

// Break into AVX2-sized pieces:
auto t = simd_chunk_n<8>(x);
```

Note that we can't create a template which takes a parameter which is either an integer or a type, so we must provide a new name for this short-hand function.

§ 2. Wording

§ 2.1. Rename `split` to `chunk` in overview

```
❶ // [simd.creation], basic_simd and basic_simd_mask creation
template<class V, class Abi>
constexpr auto
simd_split simd_chunk(const basic_simd<typename V::value_type, Abi>& x) {
template<class M, class Abi>
constexpr auto
simd_split simd_chunk(const basic_simd_mask<mask-element-size<M>, Abi>& x)
```

§ 2.2. Add `simd_chunk_n` to overview

```
❶ // [simd.creation], basic_simd and basic_simd_mask creation

template<std::size_t N, class T, class Abi>
constexpr auto simd_chunk_n(const basic_simd<T, Abi>& x) noexcept;

template<std::size_t N, std::size_t Bytes, class Abi>
constexpr auto simd_chunk_n(const basic_simd_mask<Bytes, Abi>& x) noexcept;
```

§ 2.3. Add description of `basic_simd` and `basic_simd_mask` creation [`simd.creation`]

◆ `basic_simd` and `basic_simd_mask` creation [`simd.creation`]

```
template<class T, class Abi>
constexpr auto simd_split simd_chunk(const basic_simd<typename T::value_type, Abi>& x) noexcept;

template<class T, class Abi>
constexpr auto simd_split simd_chunk(const basic_simd_mask<mask_element_type, Abi>& x) noexcept;
```

```
1) template<std::size_t N, class T, class Abi>
    constexpr auto simd_chunk_n(const basic_simd<T, Abi>& x) noexcept;

2) template<std::size_t N, std::size_t Bytes, class Abi>
    constexpr auto simd_chunk_n(const basic_simd_mask<Bytes, Abi>& x) noexcept;
```

Effects:

- For the first overload this is equivalent to calling `simd_chunk<std::resize_simd_t<N, basic_simd<T, Abi>>>(x)`
- For the second overload this is equivalent to calling `simd_chunk<std::resize_simd_t<N, basic_simd_mask<Bytes, Abi>>>(x)`